

- 1) Defina y explique el concepto de Bus de Sistema de un Procesador. Ilustre un ejemplo.
- 2) El siguiente código es ejecutado en un computador tipo ARC de 32 bits:

```
.begin
a_start .equ 9000
.org 3000
length: 20 #3000
address: a_start #3004

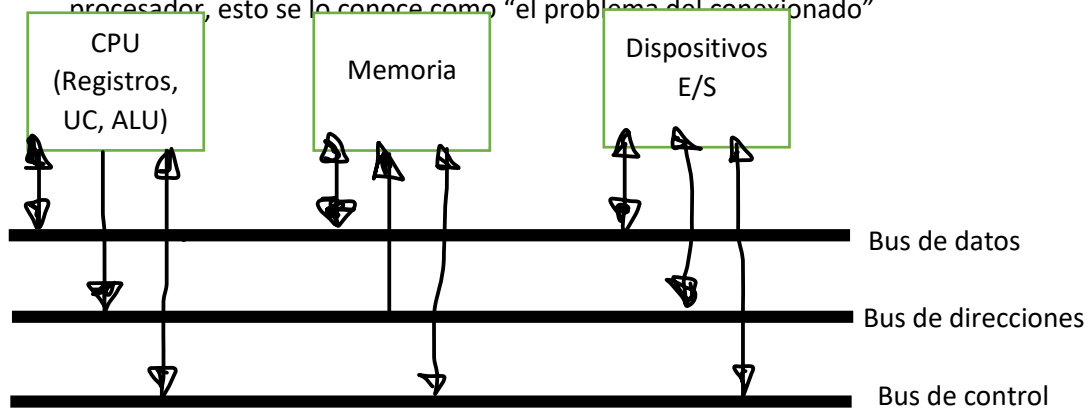
.org 1000
a: 25 #1000
.org 2048
ld [length], %r1 #2048
ld [address], %r2 #2052
ld [a], %r3 #2056
and %r3, %r0, %r4 #2060
sub %r3, %r1, %r5 #2064
and %r1, %r1, %r0 #2068
add %r1, -4, %r7 #2072
add %r1, %r3, %r6 #2076
or %r1, %r2, %r8 #2080
sub %r6, %r2, %r6 #2084
add %r15, 4, %r6 #2088

.end
```

Se pide:

- a) Determinar lo que hace el código cuando es ejecutado. Justifique.
 - b) Indique la posición en la cual se encuentra cada instrucción. Justifique.
 - c) Indique qué instrucciones son directivas para el compilador y al ensamblador.
 - d) Elabore una tabla de seguimiento de registros y variables tal que permita Indicar el valor de todos los registros (incluyendo el PC una vez ejecutada la última instrucción).
 - e) Escribir la tabla de símbolos.
-
- 3) Ilustre un trayecto de datos organizado en 1 bus, 2 buses y tres buses. Describa el funcionamiento de cada uno, compárelos identificando ventajas y desventajas.
 - 4) Explique el concepto de Macro y el concepto de subrutina. Brinde un ejemplo de implementación de cada uno.

- 1) El Bus de Sistema, es un canal de comunicación con un conjunto de conexiones por el cual los distintos componentes de un computador se traspan información. Es de utilidad, ya que sin él, cada componente iría individualmente cableado al procesador, esto se lo conoce como “el problema del conexionado”



Está conformado por:

Bus de datos:

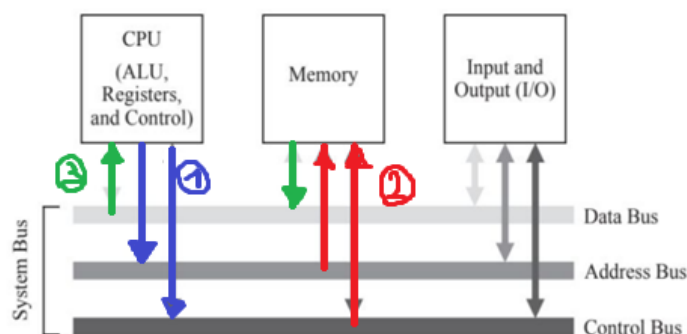
Transporta instrucciones

Bus de Direcciones:

Transporta direcciones de memoria generadas por el CPU a través del Program Counter. Es unidireccional, porque es imposible y no tiene sentido que la memoria genere direcciones.

Bus de Control: Transporta órdenes como las de lectura y escritura. A través de ella, se define la acción a realizar.

ejemplo: `ld rs1, rs2`



1) envía por bus de control la orden de lectura
envía por bus de direcciones la dirección de memoria a leerse reflejada en rs1.

2) Memoria recibe la orden de lectura en la posición de memoria rs1.

3) Luego de que la memoria lee el dato existente dentro de la dirección rs1, lo envía al CPU a través del Bus de Datos para posteriormente ser guardado en rs2.

El bus de sistema se relaciona directamente con el ciclo fetch porque:
Durante la búsqueda, el PC le indica la próxima dirección de memoria a ejecutar la siguiente instrucción. Esta dirección de memoria viaja por el bus de direcciones y es recibida por la memoria ram, a su vez, se indica a través del bus de control que la siguiente dirección de memoria debe ser leída. Al ser leída, la ram transporta la instrucción y el dato nuevamente hacia el cpu por el bus de datos.

La cpu al recibir la instrucción, la misma es alojada en el IR a la espera de ser decodificada por la Unidad de Control a través del decodificador de instrucciones durante el ciclo de decodificación.

2)a) véase tabla_de_seguimiento.xls

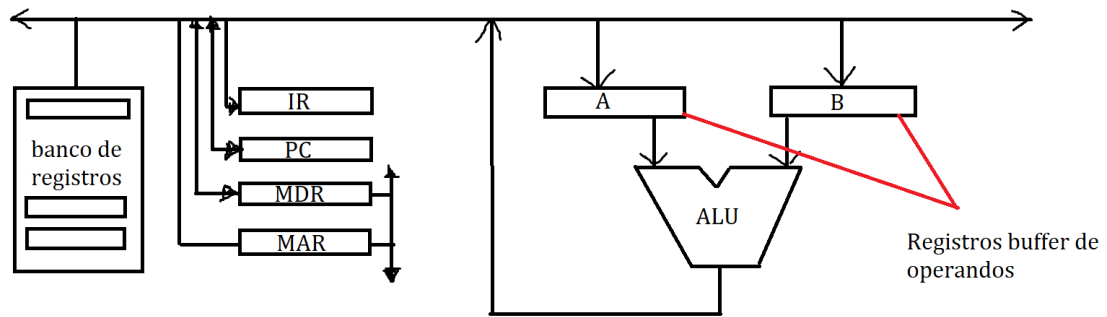
El programa comienza leyendo direcciones de memoria con ld y guarda el dato de esas direcciones de memoria en registros. Realiza operaciones aritméticas entre los registros, add, sub y operaciones or bit a bit. El programa finaliza con r6 = 4 debido a que r15 nunca fue inicializado, el simulador ARC utilizado en la materia, así lo asume debido a que comienza inicializando en 0 todos los registros generales.

b) las posiciones de memoria de aquellas instrucciones que generan código maquina, comienzan en 2048 porque así lo indica la directiva al ensamblador .org 2048

c) las directivas son, .begin, .org, .end y .equ que se utiliza para asignar un valor inmediato pero que no ocupa dirección en la memoria.

3)

Trayecto de datos organizado en 1 bus



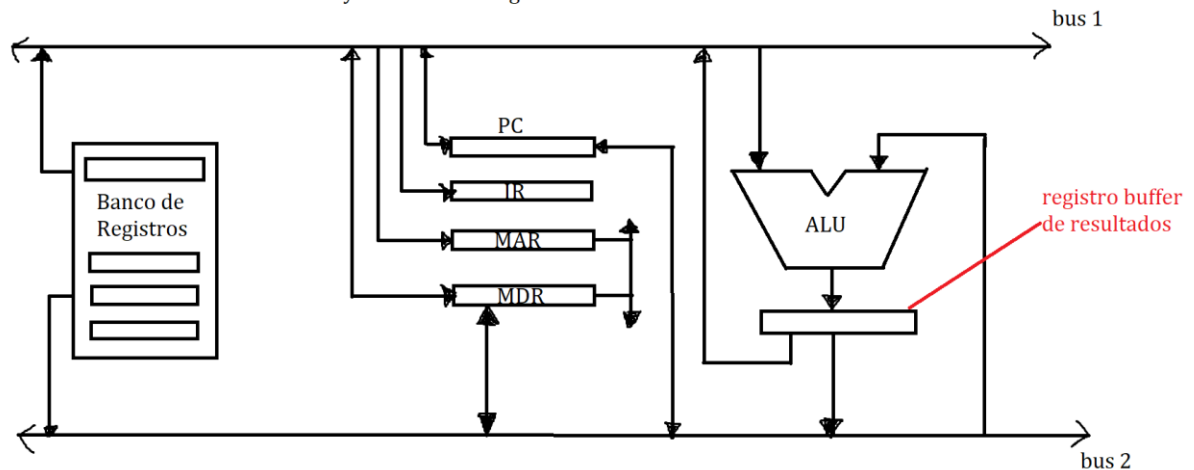
Los registros buffer son utilizados porque toda la ALU no puede recibir los dos operandos a la vez en una organización de un bus.

Utiliza 3 ciclos de reloj. Uno para insertar el rs1 en en buffer A, otro para insertar el rs2 en el buffer B, y otro para que el rsd sea guardado en el banco de registros.

La ventaja es que es considerablemente menos costoso en comparación a los otros.

La desventaja es que es mas lento.

trayecto de datos organizado en 2 buses

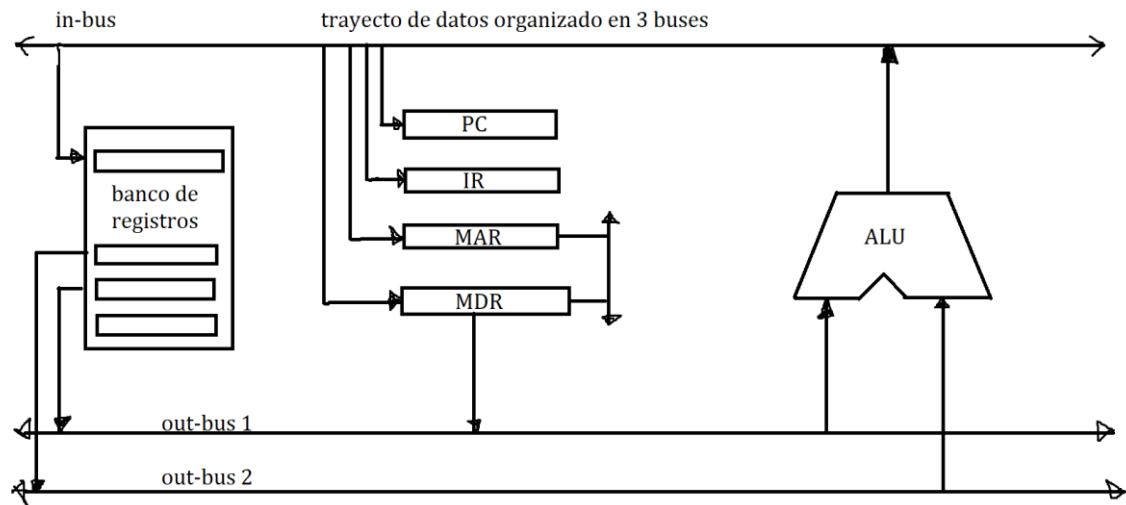


El de dos buses, hace mas eficiente la operación. Utiliza dos ciclos reloj. Uno enviar el rs1 y el rs2 a la ALU y otro ciclo reloj para que el resultado de la operación sea guardado en rsd.

Utiliza un buffer para los resultados, esperando que uno de los dos buses “se despeje” para con el segundo ciclo reloj, poder enviar la información.

Ventaja: Más rápido que la organización en 1 bus.

Desventaja: Más costoso



Finalmente, el trayecto de datos organizado en 3 buses, en situaciones idóneas, puede realizar todo el proceso en un mismo ciclo reloj. Utiliza dos buses para enviar los registros rs1 y rs2 a la ALU, y un bus para enviar el rsd al banco de registros.

La ventaja principal de esta organización, es su alta eficiencia y su veloz respuesta.

La principal desventaja es que es el más costoso de los 3.

4)

Macro:

Es accedida en tiempo de ensamblado. Sus parámetros son interpretados por el ensamblador y reemplazados por lo que corresponda en cada lugar que es invocada.

Ejemplo:

```
.begin
.org 2048
valorA: 5
valorB: 2
resultado: 0
!-----
.macro mcr_sumar var1, var2, resultado
ld [var1], %r1
ld[var2], %r2
add %r1,%r2,%r3
st %r3, [resultado]
.endmacro
!-----
main: mcr_sumar valorA, valorB, resultado
.end
```

Subrutina:

Es accedida por un CALL en tiempo de ejecución y termina con un JMPL en tiempo de ejecución. Sus parámetros le llegan en tiempo de ejecución a través de la pila o registros.

Código de máquina está alojado en una dirección de memoria específica y única.

Ejemplo:

```
.begin
.org 2048
main:
    call sbr_add
    ba final

!-----
sbr_add:  ld [x], %r1
          ld [y], %r2
          add %r1, %r2, %r3
          st %r3, [resultado]
          jmp1 %r15 + 4, %r0

!-----
x: 15
y: 5
resultado: 0
final:    ba final
.end
```